Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 70%. We keep your highest score.

Next item →

1. Alex is building a Python application that needs to test a new MCP server locally before deploying it to a remote setup. They want to quickly test tools, resources, and prompts without network overhead. Which setup should Alex use?

1 / 1 point

- ○ Use HTTP transport with a remote server URL and configure the client for async calls
- ○ Connect to multiple MCP servers simultaneously without specifying transport
- ◉ Use In-Memory Transport by passing the FastMCP server object directly to the client
- ○ Use STDIO transport with a remote HTTP URL

Nice work

In-Memory Transport allows fast, local testing when the client and server are in the same process.

2. Jason is creating a multi-server MCP application that needs to perform calculations and fetch documentation. One server will run locally using STDIO, and another will be hosted remotely via HTTP. Jordan also wants the agent to remember prior user interactions across these servers. Which approach should they follow?

1 / 1 point

- ○ Use a single HTTP server for both functionalities and avoid registering custom tools
- ○ Use HTTP transport for the local server and skip async execution
- ◉ Use MultiServerMCPClient to connect to both STDIO and HTTP servers, implement InMemorySaver for conversation memory, and load tools for the agent
- ○ Use only STDIO transport and fetch tools separately for each server manually

Nice work

MultiServerMCPClient supports multiple servers with different transports, while InMemorySaver allows the agent to maintain context across exchanges.

3. You want to create a local MCP server that can be tested using STDIO transport. Which step is essential for enabling the client to communicate with the server?

1 / 1 point

- ○ Importing StreamableHttpTransport for local server communication
- ○ Using load_mcp_tools to fetch MCP tools from the server
- ○ Creating a MultiServerMCPClient without a transport
- ◉ Writing the server content to a file named stdio_server.py

Nice work

The server content must be saved so that STDIO transport can launch it as a subprocess.

4. Which of the following correctly describes how STDIO transport is configured for a local MCP server?

1 / 1 point

- ○ Use the HTTP URL of the server as the transport parameter
- ◉ Specify the Python interpreter as the command and the server file (stdio_server.py) as args to launch the server
- ○ Provide the library name to resolve-library-id instead of the server file
- ○ Pass the server content directly to the Client constructor without writing a file

Nice work

This allows the client to spawn the server as a subprocess for STDIO communication.

5. You are building an MCP server to manage project files and documentation. One of your tasks is to create a prompt that generates a separate documentation file for an existing code file. During execution, the server needs to request the target file name and the new documentation file name from the user.

Which MCP feature would you use to accomplish this behavior?

1 / 1 point

- ○ Resources
- ◉ Context user elicitation
- ○ Tools
- ○ Prompt templates without Context

Nice work

The Context object allows the server to request structured inputs from the user during prompt building or tool execution.

6. Which of the following statements correctly differentiates between MCP tools and MCP resources?

1 / 1 point

- ○ Resources can be used to write files, while tools are limited to reading files only.
- ○ Tools are for passive access to data, whereas resources are for performing actions.
- ○ Both tools and resources are used exclusively for logging and progress reporting.
- ◉ Tools perform active operations such as creating or deleting files, whereas resources provide controlled access to data such as reading files or listing directories.

Nice work

This aligns with the definitions provided in the video.

7. Which of the following best describes the purpose of the connect_to_server method in the MCP client class?

1 / 1 point

- ○ Directly executes MCP server tools such as write_file or delete_file from the client without using handlers.
- ◉ Validates the MCP server script file type, initializes a FastMCP Client object with elicitation, progress, and message handlers, and establishes an asynchronous session with the server.
- ○ Stores all MCP server responses in a global variable for later logging and analysis.
- ○ Defines and sends prompt templates from the client to Claude without connecting to the MCP server.

Nice work

This method ensures that the script is valid, attaches handlers, and starts an async session with the MCP server.